

What is programming language?

A programming language is a formal computer language designed to communicate instructions to a machine, particularly a computer. Programming languages can be used to create programs to control the behavior of a machine or to express algorithms.

Java

Java is a multipurpose programming language and a platform independence.

Types of Java Application

There are mainly four types of application that can be created using java programming:

1. Standalone Application

It is also known as desktop application or window-based application. An application that we need to install on every machine such as media player, antivirus etc. AWT and Swing are used in java for creating standalone applications.

2. Web Application

An application that runs on the server side and creates dynamic page, is called web application. Currently servlet, jsp, struts, jsf etc. technologies are used for creating web applications in java.

3. Enterprise Application

An application that is distributed in nature, such as banking applications etc. It has the advantage of high level security, load balancing and clustering. In Java, EJB is used for creating enterprise applications.

4. Mobile Application

An application that is created for mobile devices. Currently Android and Java ME are used for creating mobile applications.

JDK, JRE, JVM, IDE?

1. JDK(Java Development Kit)

JDK is a software development environment which is used to develop java applications and applets. It physically exists. It contains JRE + development tools.

2. JRE (Java RunTime Environment)

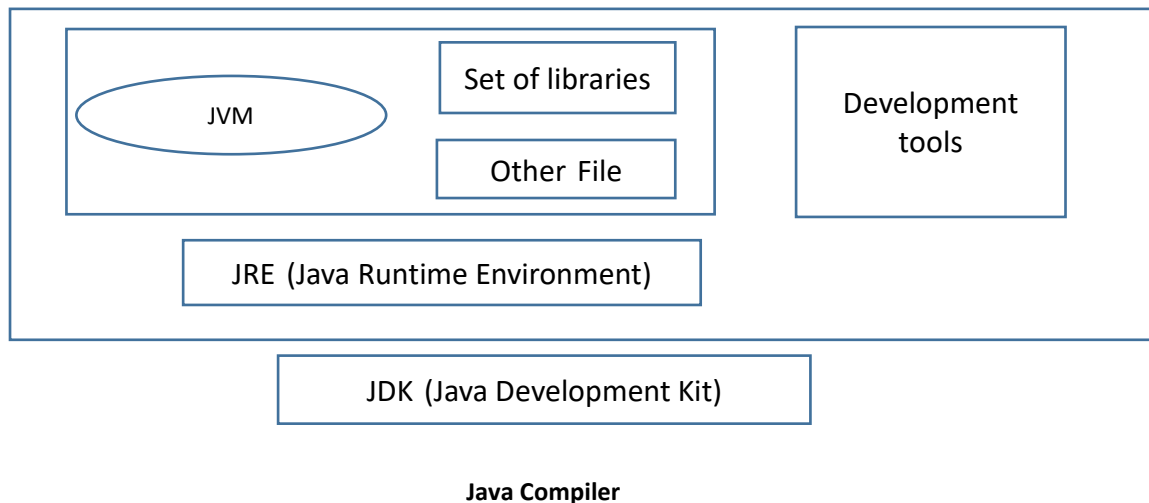
JRE is a software layer that runs on top of a computer's operating system software and provides the *class libraries* and other resources that a specific Java program needs to run.

3. Java Compiler

A Java compiler is a program that takes the text file work of a developer and compiles it into a platform-independent Java file. Java compilers include the Java Programming Language Compiler (javac)

4. JVM (Java Virtual Machine)

JVM is a specification that provides runtime environment in which java bytecode can be executed.



5. IDE (Integrated Development Environment)

An IDE (Integrated Development Environment) serves as a front end for the Java compiler and interpreter. It also includes an editor for writing Java code, a project maker, sometimes a debugger, all conveniently packaged into one program with a graphical user interface.

Sample Program of Java

```
class Sample{
    public static void main(String[] args){
        System.out.println("Hello World");
    }
}
```

Save : *Simple.java* // Save text file as the java source code file

To Compile: *javac Simple.java*//convert the source code file to class file using compiler

To Execute : *java Simple* // execute the class file

Explanation of First Java Program

class	-keyword is used to declare a class in java.
public	-keyword is an access modifier which represents visibility; it means it is visible to all.
static	-keyword, if we declared any method as static, it is known as static method. The core advantages of static method is that there is no need to create object to invoke the static method.
void	-void is the return type of the method; it means it doesn't return any value.
main	-represents startup of the program
String[] args	-is used for command line argument.
System.out.println()	-used as printed statement.

Language Fundamentals

Syntax - Syntax is about the structure or the grammar of the language. The rules that determine what is allowed are called the syntax of the language. Syntax rules specify the basic vocabulary of the language and how programs can be constructed. A syntactically correct program is one that can be successfully compiled or interpreted.

Semantic - The meaning of a program is referred to as its semantics. A semantically correct program is one that does what you want it to.

Variables - Programs manipulate data that are stored in memory. In machine language, data can only be referred to by giving the numerical address of the location in memory where it is stored. In a high-level language, such as Java, names are used instead of numbers to refer to data. A name used in this way—to refer to data stored in memory—is called a variable.

Variable names are case-sensitive. A variable's name can start with a letter, the dollar sign "\$", or the underscore character "_". Java reserved keyword can not be used as a variable name.

abstract	assert	boolean	break	byte	case
catch	char	class	const*	continue	default
double	do	else	enum	extends	false
final	finally	float	for	goto*	if
implements	import	instanceof	int	interface	long
native	new	null	package	private	protected
public	return	short	static	strictfp	super
switch	synchronized	this	throw	throws	transient
true	try	void	volatile	while	

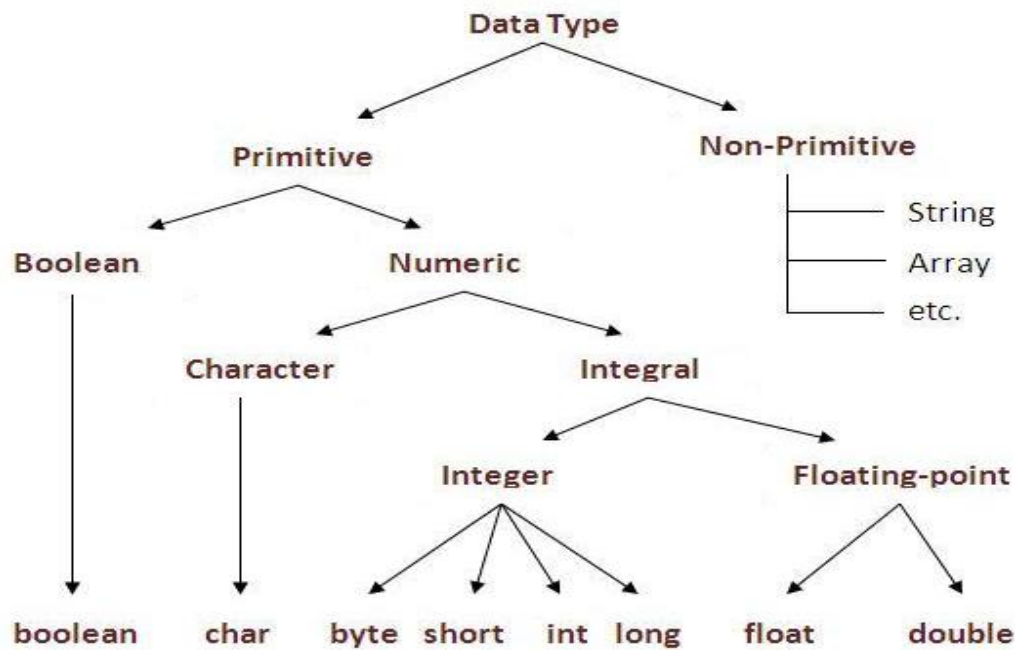
We can declare variables in several different places:

1. In a class body as class fields. Variables declared here are referred to as class-level variables.
2. As parameters of a method or constructor.
3. In a method's body or a constructor's body.
4. Within a statement block, such as inside a while or for block.

Data Types - A variable in Java is designed to hold only one particular type of data; it can legally hold that type of data and no other. The compiler will consider it to be a syntax error if you try to violate this rule. We say that Java is a strongly typed language because it enforces this rule.

In java, there are two types of data types:

- Primitive data types
- Non-primitive data types



Examples of primitive data types with initializing

```
int decimal =1230;int a= 0b 0101;int octal= 01230;int hexa= 0X FFFF;
```

<i>Primitive Data Type</i>	<i>Default Value</i>	<i>Default Size</i>	<i>Value</i>
boolean	false	1bit	true, false
char	'\u0000'	2byte	A, a, &,'+'
byte	0	1byte	-128 to 127
short	0	2 byte	-32,768 to 32,767
int	0	4 byte	- 2,147,483,648 to 2,147,483,648
long	0l	8 byte	-9,223,372,036,864,775,808 to 9,223,372,036,864,775,808
float	0.0f	4 byte	1.401298464324817E-45f to 3.402823476638528860e+38f
double	0.0d	8 byte	4.94065645841246544e-324 to 1.79769313486231570e+308

Type Casting - a way of converting data from one data type to another data type.

```
boolean variable_name=Boolean.parseBoolean(args[0]); // convert from String to boolean

char variable_name =string_variable.charAt(0); //take a character with index from the
string (character array)

byte variable_name=Byte.valueOf(args[0]); // convert from String to byte

short variable_name=Short.parseShort(args[0]); // convert from String to short

int variable_name=Integer.parseInt(args[0]); // convert from String to int

long variable_name=Long.parseLong(args[0]); // convert from String to long

float variable_name =Float.parseFloat(args[0]); // convert from String to float

double variable_name =Double.parseDouble(args[0]); // convert from String to double
```

Command Line Argument - The java command-line argument is an argument i.e. passed at the time of running the java program. The arguments passed from the console can be received in the java program and it can be used as an input.

```
public class Hello {
    public static void main (String[] args) {
        System.out.println(args[0]);
    }
}
```

Statements - Java has many kinds of statements. Most of these statements specified the evaluation of an expression. The normal flow of instructions in Java is to execute the statements of the program in sequential order from top to bottom.

```
class SampleStatement1{
    public static void main(String [] args){
        int a=1;
        System.out.println(a);
        a=2;
        System.out.println(a);
    }
}
```

Operators

A computer program is a collection of operations that together achieve a certain function. There are many types of operators.

Unary Operators - Unary operators operate on one operand. There are six unary operator.

Operator	Symbol
<i>Minus</i>	-
<i>Plus</i>	+
<i>Increment Operator</i>	++
<i>Decrement Operator</i>	--
<i>Logical Complement Operator</i>	!
<i>Bitwise Complement Operator</i>	~

Increment Operator ++

Prefix increment operator	Postfix increment operator
int x=4; ++x;	int x=4; x++;

Decrement Operator --

This operator decreases the value of its operand by one.

Prefix decrement operator	Postfix decrement operator
int x=4; int y= --x;	int x=4; int y= x--;

Arithmetic Operators - Arithmetic Operators operate on two operand. There are five arithmetic operators.

<i>Operator</i>	<i>Symbol</i>
<i>Additional</i>	+
<i>Subtraction</i>	-
<i>Multiplication</i>	*
<i>Division</i>	/
<i>Modulus</i>	%

Relational Operators - Relational Operations return a boolean value.

<i>Meaning</i>	<i>Symbol</i>
<i>Less than</i>	<
<i>Less than or Equal</i>	<=
<i>Greater than</i>	>
<i>Greater than or Equal</i>	>=
<i>Equal</i>	==
<i>Not Equal</i>	!=

Logical and Conditional Operator

<i>And</i>	&&
<i>OR</i>	
<i>NOT</i>	!
<i>Conditional</i>	? :


```

public class Conditional {
    public static void main(String[] args) {
        int num1=Integer.parseInt(args[0]);
        String result;
        result=((num1%2)==0)? "Even":"Odd";
        System.out.println(result);
    }
}

```

Shift Operator

Operator	Symbol
Left Shift Operator	<<
Right Shift Operator	>>

Assignment Operators

Operator	Symbol	Example
Right-hand side of the value assign to the left-hand side variable	=	C = A + B will assign value of A + B into C
Add the value of both sides and assign to the left-hand side variable	+=	C += A is equivalent to C = C + A
Subtract the value of both sides and assign to the left-hand side variable	-=	C -= A is equivalent to C = C - A
Multiply the value of both sides and assign to the left-hand side variable	*=	C *= A is equivalent to C = C * A
Divide the value of left-hand side by right-hand side and assign to the left-hand side variable	/=	C /= A is equivalent to C = C / A
Assign the modulus value to the left-hand side variable	%=	C %= A is equivalent to C = C % A

Operator Precedence - When two operators share an operand the operator with the higher *precedence* goes first.

Operators	Precedence
postfix	expr++ expr--
unary	++expr --expr +expr -expr ~ !
multiplicative	* / %
additive	+ -
shift	<< >> >>>
relational	< > <= >= instanceof
equality	== !=
bitwise AND	&
bitwise exclusive OR	^
bitwise inclusive OR	
logical AND	&&
logical OR	
ternary	? :
assignment	= += -= *= /= %= ^= = <<= >>= >>>=

```
class TestOperatorPrecedence{
    public static void main(String args[]){
        int num1=Integer.parseInt(args[0]);
        int num2=Integer.parseInt(args[1]);
        int num3=Integer.parseInt(args[2]);
        int num4=0;
        num4=num1-num2+num3;
        System.out.println(num4);
    }
}
```

```
        num4=num1-(num2+num3);  
        System.out.println(num4);  
    }  
}
```

Comments - Java has three kinds of comments.

1. Single Line Comment (// double- slashed)
All the text up to the end of the line is ignored.
2. Multiline Comment (begins with /* and ends with */)
All text between the two delimiters is ignored.
3. Special Comment (begins with /** and ends with */)
These are used for the javadoc system. Javadoc is used to generate API documentation from the code.

Flow Controls

if Statements - The purpose of the if statement is to make decisions, and execute different parts of your program depending on a boolean true/false value.

```
public class TestIfStatement {  
  
    public static void main(String[] args) {  
  
        int score=40;  
        if(score>=50)  
            System.out.println("PASS");  
    }  
}
```

if~else Statement - The Java if-else statement also tests the condition. It executes the if block if condition is true otherwise else block is executed.

```
class TestIfElseStatement{  
  
    public static void main(String [] args){  
  
        int score=60;  
        if(score>=50)  
            System.out.println("PASS");  
        else  
            System.out.println("FAIL");  
    }  
}
```

Nested If Statement - It is always legal to nest if-else statements which means you can use one if or else if statement inside another if or else if statement.

```
public class Test {  
  
    public static void main(String args[]) {  
        int x = 30;  
        int y = 10;  
  
        if( x == 30 ) {  
            if( y == 10 ) {  
                System.out.print("X = 30 and Y = 10");  
            }  
        }  
    }  
}
```

switch Statement - Another way to control the flow of your programs is with something called a *switch* statement. A *switch* statement gives you the option to test for a range of values for your variables.

```
class TestSwitch3{  
  
    public static void main(String args[]){  
  
        int num1=7;  
  
        int num2=6;  
  
        String operate1=args[0];  
  
        char operate2=operate1.charAt(0);  
  
        switch(operate2){  
  
            case '+': System.out.println("Addition value is "+(num1+num2));  
  
                        break;  
  
            case '-': System.out.println("Subtraction value is "+(num1-num2));  
  
                        break;  
  
            case '*': System.out.println("Multiplication value is "+(num1*num2));  
  
        }  
    }  
}
```

```

        break;

    case '/': System.out.println("Division value is "+(num1/num2));

        break;

    default: System.out.println("Operator is not suitable");

        break;

    }

}

```

for Statement - The for statement provides a compact way to iterate over a range of values. Programmers often refer to it as the "for loop" because of the way in which it repeatedly loops until a particular condition is satisfied.

```

class powerBase{

    public static void main (String[] args){

        int num1=Integer.parseInt(args[0]);

        int num2=Integer.parseInt(args[1]);

        int result=1;

        for (int i=1;i<=num2;i++){

            result*=num1;

        }

        System.out.println(result);

    }

}

```

while Statement - The *while* statement continually executes a block of statements while a particular condition is true. The while statement evaluates expression, which must return a boolean value.

```
import java.util.Scanner;

public class PrimeNum {

    public static void main(String[] args) {

        Scanner sc=new Scanner(System.in);

        System.out.print("Enter a Number: ");

        int num=Integer.parseInt(sc.nextLine());

        int divisor=2;

        while(num%divisor!=0)            divisor++;

        if(num==divisor)System.out.println("prime");

        else System.out.println("not prime");

    }

}
```

do-while Statement - do-while evaluates its expression at the bottom of the loop instead of the top. Therefore, the statements within the do block are always executed at least once.

```
import java.util.Scanner;

public class WhileTest {

    public static void main(String[] args) {

        Scanner sc=new Scanner(System.in);

        char ans;

        do{

            System.out.println("Hello....\nDo you want to continue? y/n");

            ans=sc.nextLine().charAt(0);

        }while(ans=='y');

        System.err.println("Program Exit...");

    }

}
```

Branching Statement

1. The **break** statement terminates the innermost switch, for, while, or do-while statement.
2. The **continue** statement skips the current iteration of a for, while , or do-while loop.
3. The **return** statement exits from the current method, and control flow returns to where the method was invoked.

Array

Java provides a data structure, the array, which stores a fixed-size sequential collection of elements of the same type. An array is used to store a collection of data, but it is more useful to think of an array as a collection of variables of the same type.

Declaring an Array - Declaring an array does not create an array or allocate space for its elements, the compiler simply creates an object reference. There are many ways to create an array.

```
dataType[] arrayRefVar; or
dataType arrayRefVar[];
```

```
/**
int [] array1,array2; //both of array1 and array2 are integer array.
int array1[],array2; // array1 is integer array and array2 is just the int data type.
**/
```

Instantiation of an Array

```
1. int [] array1=new int[4];
   array1[0]=2;

   array1[1]=6;

   array1[2]=7;

   array1[3]=3;

2. int [] array2={2,6,7,3};
```

Iterating over an Array

There are many ways to iterate the members of an array was to use for loop, the array's index.

- for(int i=0;i<array1.length;i++)
 System.out.println(array1[i]);
- for(String arr1:array1)
 System.out.println(arr1);
- System.out.println(array1[2]);

```

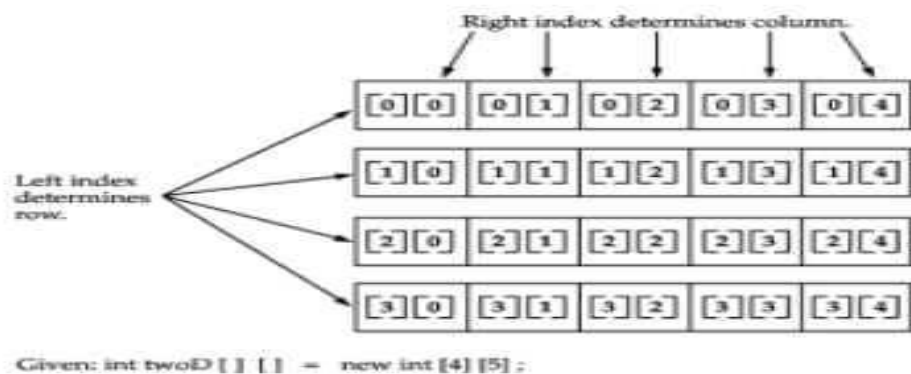
class ArrayTest{
    public static void main(String args[]){
        int [] arr1=new int[5];
        for(int i=0;i<arr1.length;i++)
            arr1[i]=i+1;
        for(int i=0;i<arr1.length;i++)
            System.out.println("Element "+i+" : "+arr1[i]);
    }
}

```

Multidimensional Array - In Java, a multidimensional array is an array whose elements are also arrays.

Declaring an Multidimensional Array

Declaring an array does not create an array or allocate space for its elements, the compiler simply creates an object reference. There are many ways to create an array.



- `int [][] array1=new int[4][3];`
`array1[0][0]=1;`
`array1[0][1]=1;`
- `int [][] array2={{1,1,1},{2,2,2},{3,3,3},{4,4,4}};`

Iterating the value from 2 dimensional array

- `System.out.println(array1[2][1]);`
- **public class** TestTwoDimentionalArray {

public static void main(String[] args) {

```
int [][] person_mark={ {50,50,50,50,50},{60,60,60,60,60},  
                        {70,70,70,70,70}};  
  
for(int i=0;i<3;i++){  
    for(int j=0;j<5;j++){  
        {  
            System.out.print(person_mark[i][j]+",");  
        }  
  
        System.out.println();  
    }  
}  
}
```